

Yazılım Testi: Dört Soru ile Test Sınıflandırması

ÖFY

September 2026

Özet

Yazılım testi tek bir liste değil, üç bağımsız eksenin bileşimidir: testin sistemin ne kadarını çalıştırdığı (boyut), testin hangi hata türünü avladığı (niyet) ve testin sistemin hangi boyutunu sınıadığı (kesit). Test türlerinin sayısı sınırsızdır; ancak bir test türü, yakaladığı hata sınıfı ilgili sistemde gerçekten yaşanmıyorsa test hattına girmeyi hak eder. Bu ilkeye dayanan bakım döngüsü, kodu önce küçültüp görünür kılar, ardından hızlı ve kararlı testlerden yavaş ve kırılğan olanlara doğru ilerler: statik denetimler, dosya bölme, loglama, teknik borç, kayıtlı yapay zekâ cevaplarına dayanan sözleşme testleri, regresyon, kullanıcı hikâyelerinden türetilen uçtan uca testler ve son olarak sistemin yapısına özgü testler. Döngünün ilk yarısı kodu değiştirdiği, ikinci yarısı bu değişikliği doğruladığı için, her döngü mevcut testlerin yeşil olduğu doğrulanarak başlar. Adımlar “ne zaman test yazılır” sorusunu cevaplar; veri, ağ, durum ve zaman gibi kesitler ise “sistemin hangi boyutu test edilir” sorusunu cevaplar ve adımların dışında, her adıma yayılarak durur. Veri kalitesi bir sayaç değil, boru hattının her kapısında ayrı bir test ailesidir; ağ bozukluğu tek bir şey değil, hattın üç kesitinde farklı sonuç doğuran altı ayrı davranıştır. Yük, kaos ve güvenlik denetimi gibi maliyetli test aileleri ilk somut arıza yaşanana kadar bilinçli olarak kapalı tutulur; erken kurulmaları, henüz var olmayan bir hataya vergi ödemek anlamına gelir. Test sayısı bir hedef değildir: her testin cevap verdiği bir “bu kırılırsa ne olur” sorusu bulunmalıdır.

Giriş

Test literatürü, yüzü aşkın adlandırılmış test türü içerir ve bu türler çoğunlukla tek bir düz liste hâlinde sunulur. Bu sunum iki farklı şeyi aynı kefeyle koyar. Birinci eksen **boyuttur**: testin sistemin ne kadar büyük bir parçasını çalıştırdığı. Birim (*unit*) testi tek bir fonksiyonu sahte bağımlılıklarla çalıştırır; entegrasyon testi birkaç parçayı en az bir gerçek bağımlılıkla birlikte koşturur; uçtan uca (*end-to-end*, *e2e*) test sistemi bütün hâlinde, kullanıcının gördüğü arayüzden yürütür. İkinci eksen **niyettir**: aynı boyuttaki bir testin farklı amaçlarla yazılabilmesi. Regresyon, smoke, yük ve mutasyon testleri katman değil, niyettir; her biri farklı bir hata sınıfını avlar.

Bu iki eksenin yanında üçüncü bir eksen daha vardır: **kesit**. Kesit, testin sistemin hangi boyutunu sınavı olduğunu söyler. Veri, ağ, durum ve zaman birer kesittir. Bir kesit tek bir adıma ait değildir; hattın her katmanında ve döngünün her adımında farklı bir testle karşılığı bulur. Örneğin ağ kesiti, birim katmanında sahte sunucuyla, sınır katmanında gerçek ağ bozucuyla, sistem katmanında tarayıcı ağ profiliyle, canlıda ise coğrafi sentetik izlemeyle test edilir. Kesit eksenini olmadan bu testler adımlara dağılır ve sistemin bir boyutunun hiç sınavı olmadığı görünmez.

Bu ayrımlar yapılmadığında iki hata ortaya çıkar. Birincisi, niyet ekseninde zengin bir test seti kurulurken boyut ekseninde bir katmanın boş bırakılmasıdır; örneğin bin beş yüz birim testi ve kapsamlı bir uçtan uca takım varken, gerçek veritabanıyla koşan orta katmanın hiç bulunmaması. Gerçek bağımlılığın gerçek zamanlamasından kaynaklanan hatalar tam bu katmanda yakalanır; birim testler sahte veritabanı kullandığı için yarışı göremez, uçtan uca testler ise yavaş olduğu için her senaryoyu koşturamaz. İkinci hata, sektörde moda olan her test türünün, sistemde karşılığı olup olmadığına bakılmadan eklenmesidir.

Bu not, bir bakım döngüsünü referans alarak hangi adımda hangi testlerin yazılacağını ve her testin ne olduğunu açıklar. Döngü şu sırayı izler: statik denetimler, dosyaların iki yüz satıra bölünmesi, loglama, teknik borç, sözleşme testleri, regresyon, kullanıcı hikâyeleri ile uçtan uca testler, sistemin yapısına özgü kalan testler. Döngünün ardından kesitler ayrı bir bölümde işlenir. Not, tek geliştiricili ya da küçük ekipli, hızlı ilerlemek zorunda olan bir bağlam için yazılmıştır; büyük kurumsal süreçlerin gerektirdiği denetimler kapsam dışı olarak ayrıca işaretlenmiştir. Örnekler, video yükleyip yapay zekâ modeliyle klip üreten bir sistemden alınmıştır; ancak ilkeler bu sistem türüne özgü değildir.

Döngünün Yapısı

Döngü, kodu değiştiren adımlarla kodu doğrulayan adımların ardışık dizilişidir. İlk üç adım (bölme, loglama, teknik borç) kodu küçültür, görünür kılar ve düzeltir; sonraki adımlar bu değişikliği doğrular. Bu sıralamanın gerekçesi, küçük dosyada borcun daha kolay görülmesi ve log varken düzeltmenin daha güvenli olmasıdır. Test adımlarının kendi içindeki sıra ise “kırılgan olandan sağlam olana” ilkesine dayanır: önce yapay zekâ ile konuşan en kırılgan uç kayıtla sabitlenir, sonra bilinen kırıklar kilitlenir, sonra kullanıcı akışları yazılır.

Döngünün yapısal zayıflığı, doğrulamanın değişiklikten sonra gelmesidir. Teknik borç adımında bir davranış bozulursa, bu ancak aynı döngünün test adımlarında görülür ve o noktada “bu hata eskiden de var mıydı” sorusu cevapsız kalır. Çözüm döngüyü bozmak değil, döngüye girmeden önce bir önceki döngünün tüm testlerini (birim, sözleşme, regresyon, uçtan uca) bir kez koşturup yeşil almaktır. İlk döngüde bu mümkün değildir; ikinci döngüden itibaren maliyetsizdir.

Döngü boyunca geçerli tek kural şudur: test sayısı hedef değildir. Her testin cevap verdiği bir “bu kırılırsa ne olur” sorusu olmalıdır; sorusu olmayan test yazılmaz. Bu kural, özellikle testlerin yapay zekâ ajanlarına yazdırıldığı durumda önem taşır. Ajanın “eksik test yaz” görevini en hızlı kapatma yolu, kodun mevcut davranışını olduğu gibi doğrulayan testler üretmektir; bu testler yeşil olur, kapsama oranını artırır, fakat kodun ne yapması gerektiğini değil ne yaptığını sınırladığı için hiçbir hatayı yakalamaz.

Kapsam Ekseninin Katmanları

Kapsam eksenini yazılımı dört katmana ayırır. Her katman, testin sistemin ne kadarını gerçek olarak çalıştırdığıyla tanımlanır.

1. **Birim:** tek bir fonksiyon ya da sınıf çalışır; her bağımlılık sahtedir. Milisaniyede koşar, binlercesi bulunur.
2. **Bileşen:** bir modül, kendi iç bağımlılıklarıyla birlikte çalışır; dış dünya (veritabanı, ağ, dosya sistemi) sahtedir. Bellek içi veritabanı ve sahte HTTP sunucusu bu katmanın araçlarıdır.
3. **Sınır (entegrasyon):** en az bir gerçek dış bağımlılık kullanılır (gerçek veritabanı, gerçek dosya sistemi, gerçek kuyruk); tarayıcı ve arayüz yoktur. Gerçek bağımlılığın gerçek zamanlamasından doğan hatalar bu katmanda yakalanır.
4. **Sistem:** her şey gerçektir; test kullanıcının gördüğü arayüzden yürütülür. Yavaş ve kırılgandır, az sayıda olur.

Bu dört katmanın dışında iki uç daha vardır; ancak bunlar katman değil, ortamdır. **Statik** uçta kod hiç koşmaz: tip denetimi, lint ve fitness function kodu okuyarak denetler. **Canlı** uçta kod üretim ortamında koşar: sentetik

izleme ve canary gerek kullanıcı trafiğinin yanında alışır. Katman, testin sistemin ne kadarını alıştırdığını söyler; ortam ise nerede alıştırdığını. Aynı katmandaki bir test farklı ortamlarda kořabilir: bir sistem testi CI’da sahne ortamına kořar da, canlıda smoke testi olarak da yrtlebilir.

Katmanlar arasındaki oran sabit değildir. Test piramidi (Cohn, 2009) ok birim, orta entegrasyon, az sistem onerir; ancak bu oran mantık ağırlıklı kod iin geerlidir. Baėlantı ağırlıklı kodda (arayz, yapıştırıcı kod, entegrasyon servisleri) bileřen ve sınır katmanları birim katmanından daha kalabalık olur. Oranı belirleyen, sistemin hangi katmanda kırıldığıdır.

Kesit Ekseni

Kesit, sistemin bir boyutudur: iinden akan veri, zerinden konuřtuėu aė, tuttuėu durum, baėlı olduėu zaman. Her kesit iin soru aynıdır: bu boyut bozulduğunda sistem ne yapıyor. Kesitler adımlara sığmaz, nk aynı kesit her katmanda farklı bir arala test edilir ve yalnızca bir adıma yerleřtirildiğinde diėer katmanlardaki testleri unutulur. Bu nedenle kesitler dngnn dıřında, ayrı bir blmde iřlenir; dngnn adımları hangi kesit testinin hangi noktada yazılacağına gnderme yapar.

Notta drt ana kesit ayrıntılı iřlenir: veri, aė, durum ve zaman. Drt ek kesit (kimlik, kaynak sınırları, yapılandırma, gzlemlenebilirlik, daėıtım, kullanıcı hatası) daha kısa tutulur; oėu ilk arıza yařandığında aılır ya da mevcut adımlara birer paragraf olarak eklenir.

Adım 0: Döngüye Girmeden

Bu adımdaki denetimler her koşulda otomatik olarak ve sürekli açık durur. Ortak özellikleri, bir kez kurulduktan sonra sıfır bakım maliyetiyle çalışmaları ve yakaladıkları hata sınıfının kesin olarak yaşanmasıdır.

Tip Denetimi

Tip denetimi (*type checking*), kodun derlenmeden önce her değerin beklenen türde olup olmadığını kontrol eder. En ucuz test türüdür; binlerce hatayı, kod hiç çalıştırılmadan ve ek yazım gerektirmeden yakalar. TypeScript için **tsc**, Go için derleyicinin kendisi bu işi görür.

Lint

Lint, kodu çalıştırmadan okuyup bilinen hata kalıplarını ve stil bozukluklarını yakalayan araçtır. Tipik yakaladıkları: tanımlanmış ama hiç kullanılmayan değişken, **await** unutulmuş asenkron çağrı, **return** sonrasında kalan ulaşılamaz kod, **==** yerine **===** olması gereken karşılaştırma, boş **catch** bloğu. Tip denetiminden farkı şudur: tip denetimi “bu değer metin mi sayı mı” der, lint “bu kod şüpheli görünüyor” der. İkisi birbirini tamamlar. JavaScript için **eslint**, Go için **golangci-lint**, Python için **ruff** yaygın araçlardır. Editörde yazarken uyarı verir, sürekli entegrasyon (*continuous integration*, CI) hattında build’i kırar.

Fitness Function

Fitness function, kodun yapısına dair kuralları otomatik olarak denetleyen test-tir. Kodun ne yaptığına değil, nasıl kurulduğuna bakar. Ford, Parsons ve Kua (2017), terimi mimari kararların zamanla aşınmasını önleyen otomatik bekçiler için kullanmıştır. Örnek kurallar: hiçbir dosya iki yüz satırı geçemez; arayüz katmanı veritabanı katmanını doğrudan içe aktaramaz; her servis dosyasında log çağrısı bulunmalıdır; belirli bir klasördeki dosya sayısı otuzu aşamaz.

Çalışma şekli basittir: bir betik kodu tarar, kuralı bozan dosya varsa CI kırmızı olur. Adın gerekçesi, mimariye bir “form” tanımlanması ve kod bu formdan saptığında alarm çalmasıdır. Bir kez yazılır, sonra süresiz bekçilik yapar; geliştirici kuralı unutsa da kural unutulmaz. Satır limiti betikleri ve sapma sayaçları bu ailenin üyeleridir; çoğu ekip bu katmanı adını koymadan kurmuş olur.

Flakiness Takibi

Aynı test, aynı kodla bazen geçip bazen kalıyorsa buna titrek (*flaky*) test denir. Sebepler üç gruba ayrılır: zamanlama (test bir saniye bekler, işlem bazen bir saniyeden uzun sürer), sıra (paralel koşan testler birbirinin verisini bozar) ve dış bağımlılık (ağ, sistem saati, rastgele sayı üretici). Fowler (2011) bu sınıfı

“deterministik olmayan test” adıyla ele almış ve ortak sebebin, testin kontrol etmediği bir şeye bağlı olması olduğunu belirtmiştir.

Titrek test en tehlikeli testtir; mekanizması şöyle işler: bir süre sonra herkes “o zaten bazen kalır” deyip kırmızıyı görmezden gelir, o gün gerçek bir hata girer ve kimse bakmaz. Takip için CI’da her testin geçme ve kalma geçmişi tutulur; son yirmi koşuda üç kez sebepsiz kalan test listeye düşer. Listedeki test ya düzeltilir ya silinir, “tekrar dene” ile geçirilmez. Playwright ve Vitest gibi araçlar kalan testi otomatik olarak yeniden koşturabilir (**retries** ayarı); bu ayar açıkken titreklik gizlenir. Ayarı kapalı tutup listeyi görmek daha doğru yaklaşımdır.

Adım 1: Dosya Bölme

İki yüz satırı aşan dosyaların parçalanması mantık deęiřtirmez, kod taşır; bu nedenle yeni test yazılmaz. Tek istisna, testi hiç bulunmayan bir dosyanın bölünmesidir. Bu durumda bölmeden önce dosyaya birkaç örnek girdi verilir ve verdięi çıktı kaydedilir. Bölme işleminden sonra aynı girdiler tekrar verilir; çıktı kayıtları aynıysa davranış korunmuştur.

Bu teknik literatürde üç adla anılır ve ayırım yalnızca vurgudur. **Snapshot** ya da **golden master**, çıktının kayıtlarıyla karşılaştırılması tekniğinin adıdır. **Characterization** testi, Feathers'ın (2004) miras kod için önerdięi yaklaşımdır: kodun ne yaptığını, doğru olup olmadığını bilmeden kayda almak ve deęişiklik sonrasında aynı davranmaya devam ettiğini sınamak. **Diferansiyel** test ise kayıt dosyası yerine eski ve yeni kodu yan yana tutup aynı girdiyle canlı karşılaştırma yapar. Bölme adımı için pratik özet şudur: bölmeden önce cevapları kaydet, böldükten sonra karşılaştır. Karşılaştırma tuttukten sonra kayıt silinebilir ya da kalıcı test olarak tutulabilir.

Characterization testi ile veri kesitindeki altın veri seti arasındaki fark, kaydın ne anlama geldiğidir. Characterization kaydı “kod şu an ne yapıyor” der, doğru olup olmadığı bilinmez. Altın set kaydı “kod ne yapmalı” der, birisi elle doğrulamıştır. İkisi aynı mekanizmayı kullanır ama farklı soruya cevap verir.

Adım 2: Loglama

Eksik log çağrılarının tamamlanması adımımda test yazılmaz. Log, davranış değil gözlemdir; yanlış bir log satırı sistemin çıktısını değiştirmez. Loglamanın test hattındaki karşılığı, sonraki adımlarda arıza enjeksiyonu ve sentetik izleme testlerinin doğru logu üretip üretmediğinin dolaylı olarak görülmesidir.

Bu kural log içeriği içindir; logun varlığı test edilebilir. Her hata yolunun en az bir log satırı üretip üretmediği fitness function ile denetlenir; istek kimliğinin hattın başından sonuna taşınıp taşınmadığı ise gözlemlenebilirlik kesitinde işlenir.

Adım 3: Teknik Borç

Teknik borç düzeltmeleri mantığa dokunur; dolayısıyla bu adımda yazılan testler, döngünün regresyon setinin ilk üyeleridir. Aşağıdaki teknikler bir arada “hangi girdi” sorusunu cevaplar; hataların büyük çoğunluğunu bulan, test türü seçimi değil girdi tasarımıdır.

Düzeltilme Öncesi Kırmızı Birim Testi

Bir borç düzeltilmeden önce, o borcun yol açtığı yanlış davranışı gösteren test yazılır. Test kırmızı olur. Sonra borç düzeltilir, test yeşile döner. Böylece hem düzeltme kanıtlanır hem de hatanın geri gelmesi engellenir. Düzeltmeden önce de yeşil olan bir test hiçbir şeyi kanıtlamaz.

Sınır Değer Analizi

Hatalar kenarlarda oluşur. Bir fonksiyon “normal” değerle çalışır; boş liste, null, sıfır, bir, izin verilen en büyük değer ve en büyükten bir fazlası verildiğinde patlar. Sınır değer analizi (*boundary value analysis*), bu kenarların her birini teker teker denemektir. Veri kesitindeki girdi sınır testleri (çok büyük dosya, boş dosya) bu tekniğin dosya düzeyine taşınmış hâlidir.

Eşdeğerlik Sınıfları

Olası girdi sayısı milyonlarla ölçülse de girdilerin çoğu aynı davranır. “1–100 arası geçerli, sıfır ve altı geçersiz, 101 ve üstü geçersiz” kuralı üç eşdeğerlik sınıfı (*equivalence class*) tanımlar. Her sınıftan bir temsilci yeterlidir: 50, –5, 150. Milyon test yerine üç test.

Karar Tablosu

Birden fazla koşul bir arada davranışı belirliyorsa kombinasyonlar tabloya dökülür. “Kullanıcı yönetici mi $\times$ kredisi var mı $\times$ dosya büyük mü” üç koşul, sekiz kombinasyon ve her kombinasyon için bir beklenen sonuç demektir. Tablo yazıldığında unutulmuş kombinasyon görünür hâle gelir; karar tablosunun (*decision table*) asıl değeri budur. Ağ kesitindeki test matrisi (bozukluk türü $\times$ hattaki yer) bir karar tablosudur.

Durum Geçiş

Durumları olan her nesne için (taslak $\rightarrow$ gönderildi $\rightarrow$ onaylandı $\rightarrow$ yayınlandı) iki tür test yazılır: her izinli geçiş (taslaktan gönderildi olabiliyor mu) ve her yasak geçiş (taslaktan doğrudan yayınlandı olamamalı). Yasak geçişlerin test edilmesi genellikle ihmal edilir; hataların çoğu oradadır.

İdempotency

Aynı istek iki kez gönderildiğinde sonuç bir kez gönderilmiş gibi olmalıdır. Ödeme düğmesine çift tıklandığında iki kez ücret çekilmemeli; ağ kopup istek yeniden gönderildiğinde iki kayıt oluşmamalıdır. İdempotency testi, işlemi bilerek iki kez çağırıp durumun değişmediğini doğrular. Determinizmden farkı şudur: determinizm çıktının aynı olmasını ister, idempotency yan etkinin tekrarlanmamasını ister. Bir işlem deterministik olup idempotent olmayabilir (her seferinde aynı kaydı bir kez daha yazar) ve tersi de mümkündür.

Önbellek Doğruluğu

Önbellekten (*cache*) dönen cevap ile önbelleksiz taze cevap aynı olmalıdır. Test iki adımlıdır: veriyi değiştir, önbelleğin güncellenip güncellenmediğine bak. Eski değeri döndüren önbellek, kullanıcının gördüğü en kafa karıştırıcı hata sınıflarından birini üretir.

Saat ve Saat Dilimi

Sistem sahte saatle koşturulur: yaz saati geçiş gecesi, farklı saat dilimindeki kullanıcı, geri alınmış sistem saati. “Bugün” hesabı, “son yirmi dört saat” filtresi, süresi dolan yetki belirteci (*token*) gibi işler bu testlerle kırılır. Bu hata sınıfı kolay sızar, zor yakalanır; gerçek ortamda yılda yalnızca iki gece görünür. Zaman kesiti bu alt başlığın genişletilmiş hâlidir.

Sanitizer ve Race Detector

Dil seviyesinde çalışan izleme araçlarıdır. Go’da `go test -race`, iki goroutine’in aynı veriye eşzamanlı yazdığını yakalar; C ve C++’ta sanitizer araçları bellek hatalarını bulur. Geliştirici yeni test yazmaz; mevcut testleri bu bayrakla koşturur, araç arka planda izler. JavaScript’te karşılığı zayıftır.

Hata Yolu Testi

Her fonksiyonun başarılı yolu test edilir, hatalı yolu unutulur. Veritabanı düştüğünde, dosya bulunmadığında, ağ koptuğunda fonksiyon ne yapar: doğru hatayı mı fırlatır, yoksa sessizce yanlış sonuç mu döndürür. Teknik borç çoğunlukla bu yollarda saklanır; boş `catch` blokları ve yutulan hatalar tipik örneklerdir.

Eşzamanlılık Birim Testi

Aynı fonksiyon aynı anda on kez çağrılır. Sayaç, defter, kuyruk gibi paylaşılan yapılar doğru mu kalır. Race detector bu sınıfı araçla, eşzamanlılık testi elle bulur. Kredi defterlerinde yaşanan yarış hataları tam bu sınıfa girer.

Kaynak Temizliđi

Açılan bağlantı, dosya tanıtıcısı ve zamanlayıcı kapanıyor mu. Test: işlem yüz kez çalıştırılır, açık kaynak sayısının artıp artmadığına bakılır. Bu, saatlerce süren soak testinin küçültölmüş hâlidir; sızıntıyı saatler yerine saniyeler içinde gösterir.

Adım 4: Mirror Testleri

Yapay zekâ modeliyle konuşan uçlar, sistemin en kırılgan yerleridir: model sağlayıcısı modeli sessizce güncelleyebilir, cevap biçimi değişebilir, cevap yarım gelebilir. Bu adımdaki testlerin ortak ilkesi, gerçek model çağrısının test içinde bulunmamasıdır. Cevap bir kez alınıp kaydedilir (*fixture*); testler o kayıt üzerinde koşar. Böylece testler hızlı, ücretsiz ve her koşuda aynı olur.

Sözleşme

Sözleşme (*contract*) testi, iki tarafın anlaştığı biçimin bozulup bozulmadığını sınar. Modelden hangi alanların hangi adla ve hangi tipte beklendiği yazılı hâle getirilir; test bu sözleşmeyi kontrol eder. Alan adı `clip.start` iken `clipStart` gelirse test kırmızı olur. Bu sözleşme dış tarafla (model) yapılır; hattın iç aşamaları arasındaki sözleşme veri kesitinde işlenir.

Fixture Parser Testi

Gerçek model cevabı bir kez alınır, dosyaya kaydedilir. Ayrıştırıcı (*parser*) o dosyayla test edilir: “bu kayıttan üç klip çıkmalı, ilkinin başlangıcı on ikinci saniye olmalı.” Model çağrısı yoktur.

Şema Testi

Fixture dosyasının JSON şemasına uyup uymadığı denetlenir: zorunlu alanlar var mı, tipler doğru mu, fazladan bilinmeyen alan var mı. Sözleşmenin otomatikleştirilmiş hâlidir. Bu test model cevabına bakar; kullanıcı verisine bakan karşılığı veri kesitindeki giriş doğrulamadır.

Bozuk Fixture

Kayıt bilerek bozulur: bir alan silinir, boş metin konur, JSON yarıda kesilir. Ayrıştırıcı çökmeden düzgün hata veriyor mu. Modeller zaman zaman bu biçimde cevap döndürür; sistem yıkılmamalıdır. Ağ kesitinde bu testin bir türevi vardır: fixture’ın kendisi değil, gelişi bozulur (gecikmeli, yarım, kesik).

Parite

Aynı girdi iki farklı yerde aynı sonucu veriyor mu. TypeScript ayrıştırıcısı ile Python ayrıştırıcısı aynı fixture’dan aynı klipleri çıkarıyor mu; yerel ortam ile canlı ortam aynı cevabı veriyor mu. Parite, diferansiyel testin çok ortamlı özel hâlidir.

AI Eval (Kurallı)

Gerçek modele sabit girdi verilir, çıktı mekanik kurallarla kontrol edilir: “en az üç klip olmalı”, “hiçbir klip altmış saniyeyi geçmemeli”, “toplam süre video-dan uzun olamaz.” Kural mekaniktir, yargı içermez. Model sağlayıcısı modeli sessizce güncellediğinde kalite düşüşünü bu testten başkası yakalayamaz. Veri kesitindeki canlı kayma izleme, bu testin gerçek trafik üzerinde sürekli koşan hâlidir.

AI Eval (Yargıçlı)

Kuralla ölçülemeyen nitelikler için (“bu özet iyi mi”) çıktı ikinci bir modele verilir ve puanlaması istenir. Pahalı ve yavaştır; gece bir kez koşar, puan düşerse alarm verir. Yargıç modelin kendi tutarsızlığı bir sınırlılıktır; puanların tek koşuda değil eğilim olarak okunması gerekir.

Metamorfik Test

Doğru cevabın bilinmediği yerde girdi-çıkı ilişkisi sınanır (Chen, Cheung ve Yiu, 1998). Video iki kez üst üste eklendiğinde klip sayısı iki katına çıkmalı; ses seviyesi değiştirildiğinde klip zamanları değişmemeli. Cevap değil, ilişki test edilir. Yapay zekâ çıktıları için özellikle uygundur, çünkü “doğru özet” tanımlanamaz ama “girdi değişince özetin nasıl değişmesi gerektiği” tanımlanabilir. Metamorfik ilişki, veri kesitindeki çıkış değişmezlerinin girdiye bağlı biçimidir.

Adım 5: Regresyon

Regresyon Testi

Bir hata bulunup düzeltildiğinde, o hatayı yeniden üreten bir test yazılır ve kalıcı olarak bırakılır. Aynı hata geri gelirse test kırmızı olur. Kural kesindir: test hatayı gerçekten üretmelidir; düzeltmeden önce koşturulduğunda kırmızı, sonra yeşil olmalıdır. Her iki durumda da yeşil olan test regresyon testi değildir. Zamanla biriken regresyon seti, sistemin gerçekten yaşadığı hataların haritasıdır ve en değerli test setine kendiliğinden dönüşür. Veri kesitindeki altın veri seti, regresyon setinin veri tarafıdır: canlıda yaşanan her veri hatası, sebep olan girdiyle birlikte sete eklenir.

Property-Based Test

Geliştirici örnek girdi değil kural yazar: “sıkıştır sonra aç, her zaman aynı çıkmalı.” Araç binlerce rastgele girdi üretir ve kuralı bozan bir girdi arar; bulursa en küçük bozucu girdiyi verir. Claessen ve Hughes’un (2000) QuickCheck aracıyla yaygınlaşmıştır. Elle düşünülemeyecek girdileri bulması, fuzz testinden farkı ise rastgele girdiyi bir değişmeze (*invariant*) karşı sınamasıdır.

Karakter İşkencesi

Metin kabul eden her alana kötü metin verilir: emoji, sağdan sola yazılan Arapça, üç yüz karakterlik dosya adı, tırnak işaretleri, `<script>` etiketi, sıfır genişlikli boşluk. “Big List of Naughty Strings” adlı açık kaynak liste bu amaçla hazırlanmıştır; dosya adı, kullanıcı adı, başlık gibi alanlara toptan verilir. Sistem çökmemeli ve veriyi bozmamalıdır.

Mutasyon Testi

Testlerin testidir (DeMillo, Lipton ve Sayward, 1978). Araç koda küçük hatalar sokar: `>` yerine `>=`, `+` yerine `-`, bir satırın silinmesi. Her hata (*mutant*) için testler koşturulur. Test kırmızı olduysa mutantı yakalamıştır; yeşil kaldıysa yakalayamamıştır. Yakalanmayan mutant sayısı yüksekse testler sahtedir: koşar ama bir şey kontrol etmez. Yapay zekâ ajanlarına yazdırılan regresyon setinin gerçek yakalama gücünü ancak mutasyon testi gösterir.

Adım 6: Kullanıcı Hikâyeleri ve Uçtan Uca Testler

Kullanıcı Hikâyelerinin Çıkarılması

Kullanıcı hikâyesi (*user story*), sistemin bir kullanıcı için yaptığı tek bir işin kısa tanımıdır: “kullanıcı video yükler, klipler üretilir, indirir.” Hikâyeler bu adımda çıkarılır ve uçtan uca testlerin giriş noktasını oluşturur. Her hikâye en az bir kabul testine dönüşür; hikâyesi olmayan uçtan uca test yazılmaz, çünkü neyi doğruladığı belirsiz kalır.

Kabul Testi

Kabul (*acceptance*) testi, kullanıcı hikâyesi ne diyorsa onu yapar. “Kullanıcı video yükler, klipler üretilir, indirir” hikâyesi için test aynen bu adımları yürür. Hikâye geçti mi geçmedi mi; başka soru sormaz.

Uçtan Uca Test (Yapay Zekâsız)

Tarayıcıdan gerçek tıklama yapılır, ancak model çağırısı yerine kayıtlı fixture döner. Hızlı, ücretsiz ve her koşulda aynıdır; varsayılan mod budur.

Uçtan Uca Test (Tam)

Aynı testler gerçek modelle koşturulur. Yavaş ve ücretlidir; gece ya da sürüm öncesi çalışır ve kaç çağrı yapıldığını, kaç dolar harcandığını raporlar. Modelin gerçekten çalıştığını yalnızca bu mod gösterir.

Smoke Testi

Uçtan uca setten en kritik üç ila beş test seçilir: giriş yapılıyor mu, yükleme çalışıyor mu, ana iş tamamlanıyor mu. Her dağıtım (*deploy*) sonrasında canlı ortamda koşar, bir dakika sürer. Geçmezse dağıtım geri alınır. Smoke testi olmadan dağıtım yapmak, sistemin ayakta olup olmadığını kullanıcıdan öğrenmek demektir.

Sanity Testi

Küçük bir değişiklik sonrasında yalnızca ilgili alanın uçtan uca testleri koşturulur, tamamı değil. Hız için yapılan bir daraltmadır.

Sistem Testi

Uçtan uca testin tarayıcısız hâlidir: API’ye istek atılır, cevap kontrol edilir. Arayüz kırılganlığı taşımadığı için daha hızlıdır; buna karşılık arayüz hatasını görmez.

Görsel Regresyon

Her ekranın görüntüsü kaydedilir; sonraki koşuda yeni görüntüyle piksel düzeyinde karşılaştırılır. Kaymış düğme, bozulmuş renk, taşan yazı fark olarak yakalanır. Kırılabilirliği yüksektir; meşru tasarım değişiklikleri de kırmızı üretir.

Erişilebilirlik

Fare olmadan yalnızca klavyeyle gezilir: her yere ulaşılıyor mu. Ekran okuyucu her düğmenin adını okuyor mu. **axe** gibi araçlar bu denetimin büyük kısmını otomatik yapar.

Yerelleştirme

Dil değiştirildiğinde çeviri düğmeye sığıyor mu, uzun Almanca kelimeler taşıyor mu, tarih “03.09.2026” mı “9/3/2026” mı, para “1.234,56” mı “1,234.56” mı gösteriliyor.

Uyumluluk

Aynı uçtan uca testler Chrome, Firefox ve Safari’de; Windows ve Mac’te koşturulur. Birinde çalışıp diğerinde bozulan davranış bulunur.

Keşif Testi

Senaryo yoktur. Özellik bittiğinde beş dakika elle kurcalanır: tuhaf sırayla tıklamak, geri tuşuna basmak, iki sekme açmak. Otomatik testin düşünmediğini insan bulur; hızlı geliştirici için en verimli test türüdür. Keşifte en sık bulunan kullanıcı hataları (çift tıklama, geri tuşu, sayfa yenileme) otomatik uçtan uca sete alınır; bu, kullanıcı hatası kesitinde işlenir.

Yapılandırılmış Keşif

Keşfin düzenli hâlidir (Bach, 2000): otuz dakikalık süre, tek bir hedef (“yüklemeye ekranı kır”), bulguların not alınması. Ekip büyüdüğünde keşfin kayıt altına alınmış biçimidir.

Doküman Testi

README temiz bir makinede (yeni sanal makine, Docker) adım adım harfiyen uygulanır. Genellikle çalışmaz: eksik adım, eski komut, unutulmuş ortam değişkeni çıkar. Dokümanın regresyon testidir; yeni gelen kişinin ilk gününü sınar.

Adım 7: Sistemde Karşılığı Olan Testler

Bu adımdaki testler yalnızca sistemde ilgili yapı bulunuyorsa yazılır: dış API varsa sözleşme, roller varsa yetki matrisi, kiracılar varsa izolasyon.

Entegrasyon

Gerçek veritabanı ve gerçek dosya sistemi kullanılır, ancak tarayıcı yoktur. Fonksiyon doğrudan çağrılır, veritabanında doğru kaydın oluşup oluşmadığına bakılır. Birim testinin sahte veritabanı nedeniyle göremediği ve uçtan uca testin yavaşlığı nedeniyle koşmadığı hataları bu katman yakalar. Özellikle yarış ve zamanlama içeren işler (defter, kuyruk, eşzamanlı yazma) buraya aittir. Tek bir Docker konteyneriyle kurulur.

Yetki Matrisi

Satırlar roller (misafir, kullanıcı, yönetici), sütunlar uç noktalar (/upload, /admin/users), her hücrede beklenen cevap (200 ya da 403). Test tabloyu otomatik tarar. Yeni uç nokta eklenip tabloya satır eklenmezse test kırılır; unutulmuş yetki kontrolü böyle yakalanır. Yetki matrisi “kim ne yapabilir” sorusunu cevaplar; “kimlik değişince ne olur” sorusu kimlik kesitinde işlenir.

Kiracı İzolasyonu

A şirketinin kullanıcısı olarak giriş yapılır, B şirketinin verisine her yoldan ulaşmaya çalışılır: kimliği URL’ye yazmak, API’ye doğrudan istek atmak, arama yapmak. Hiçbiri veri döndürmemelidir. Çok kiracılı (*multi-tenant*) sistemlerde en pahalı hata sınıfıdır.

Faturalama Değişmezi

Kural: krediler asla kaybolmaz, asla çoğalmaz. Her yol denenir: normal işlem, iptal, yarıda hata, yeniden deneme, çift tıklama. Sonunda defter toplamı tutmalıdır. Property-based testle birleştirildiğinde güçlüdür; rastgele işlem dizileri üretilir ve toplam değişmezi her dizide kontrol edilir. Veri kesitindeki çıkış değişmezleri aynı mantığı veri için uygular: orada para değil kayıt korunur.

Bozuk Medya Korusu

Kasıtlı bozuk dosyalardan oluşan bir klasör: yarım mp4, başlığı bozuk mxf, sıfır baytlık dosya, uzantısı mp4 ama içeriği metin olan dosya. Hepsisi boru hattına verilir. Sistem düzgün hata vermeli, çökmemeli ve takılı kalmamalıdır. Medya işleyen bir sistemde fuzz testinin en isabetli hâlidir. Korusu, veri kesitindeki bozuk girdi testlerinin kalıcı deposudur; bulunan her yeni bozuk dosya korusa eklenir.

Zip-Bomb ve Zip-Slip

Zip-bomb, bir kilobaytlık arşivin açıldığında on gigabayt olması ve diski doldurmasıdır. Zip-slip, arşiv içindeki dosya adının `../../etc/passwd` gibi bir yol içermesi ve açıldığında sistem dosyasının üzerine yazmasıdır. İkisi de reddedilmedir. Zip-bomb, veri kesitindeki “çok büyük girdi” sınıfının özel bir üyesidir: küçük görünüp büyük açılan girdi.

Migrasyon ve Geri Dönüş

Eski şemadaki gerçeğe benzer veriyle başlanır. Şema yükseltilir, veri sağlam mı. Geri alınır, veri yine sağlam mı. Şema değişikliği veri kaybının birincil sebebidir.

Yedekten Dönüş

Yedek almak test değildir; geri yüklemek testtir. Yedek boş bir ortama yüklenir, sistem çalışıyor mu, veri tam mı. Yedeklerin önemli bir kısmı ilk gerçek ihtiyaçta bozuk çıkar.

Benchmark

Kritik bir işlemin süresi ölçülür ve sayıya bağlanır: “birleştirici yüz klip için iki saniyeden az.” Her koşuda ölçülür; süre eşiği aştığında kırmızı olur. Yavaşlama böylece hata gibi yakalanır.

Arıza Enjeksiyonu

Tek bir şey bilerek bozulur: disk doldurulur, ağ beş saniye geciktirilir, DNS kapatılır, tek bir dış çağrı zaman aşımına uğratılır. Sistemin tepkisi gözlenir: düzgün hata mı, sonsuz bekleme mi, yarım kayıt mı. Kaos mühendisliğinin ince ayarlı hâlidir. Ağ kesitindeki sınır katman testleri, arıza enjeksiyonunun ağ odaklı biçimidir.

Kombinatoryal (Pairwise) Test

Ekranda sekiz ayar, her biri üç seçenekli: 6561 kombinasyon. Tamamı test edilemez. Pairwise yaklaşımı, her iki ayarın her kombinasyonunun en az bir kez görülmesini sağlayan küçük bir küme seçer; matematik bunu yaklaşık on beş teste indirir. Kuhn ve arkadaşlarının (2004) çalışması, gerçek hataların büyük çoğunluğunun en fazla iki parametrenin etkileşiminden çıktığını göstermiştir.

Model Tabanlı Test

Sistemin durumları ve geçişleri çizilir (taslak → işleniyor → bitti → indirildi). Araç bu haritadan yüzlerce geçerli yol üretir ve koşturur. Geliştirici senaryo değil harita yazar.

Kullanılabilirlik

Gerçek bir insan oturtulur, görev verilir (“bu videoyu yükle, klip al”), izlenir ve müdahale edilmez. Nerede takıldığı, neyi bulamadığı kaydedilir. Otomatik test “çalışıyor” der; bu test “anlaşıyor mu” der.

Sentetik İzleme (Watchdog)

Canlı sistem her beş dakikada bir sahte kullanıcı gibi dürtülür: giriş yap, küçük bir video yükle, sonuç geldi mi. Gelmezse alarm çalar. Kullanıcı şikâyet etmeden önce arıza öğrenilir. İzleme farklı coğrafyalardan ve farklı ağ kalitelerinden yapılırsa, kullanıcının gerçekten gördüğü ağ koşulu da ölçülmüş olur.

Adım 8: İlk Arıza Yaşanınca Açılan Testler

Bu ailedeki testler kurulum ve bakım maliyeti yüksek olduğu için, yakaladıkları hata sınıfı ilk kez somut olarak yaşanana kadar bilinçli olarak kapalı tutulur. Erken kurmak, henüz var olmayan bir hataya vergi ödemektir.

Performans Ailesi

- **Load:** beklenen kullanıcı sayısı ile sistem yüklenir (yüz kişi aynı anda yükleme yapıyor). Cevap süresi kabul edilebilir mi, hata var mı.
- **Stress:** yük kırılana kadar artırılır. Kaç kullanıcıda çöktüğü, nasıl çöktüğü (yavaşlama mı, ani patlama mı) ve sonrasında toparlanıp toparlanmadığı gözlenir.
- **Soak:** orta yük saatlerce (sekiz ila yirmi dört saat) sabit tutulur. Bellek yavaş yavaş artıyor mu, bağlantılar birikiyor mu. Sızıntılar ancak zamanla görünür.
- **Spike:** sıfırdan aniden bin kullanıcı, sonra sıfır. Ani patlama kaldırıyor mu, sonrasında normale dönülüyor mu.
- **Ölçeklenebilirlik:** sunucu iki katına çıkarılır. Kapasite de iki katına çıkıyor mu, yoksa bir yerde darboğaz (tek veritabanı, tek kuyruk) var mı.

Dayanıklılık Ailesi

- **Kaos:** canlı ortamda bilerek servis öldürülür, sunucu kapatılır, ağ kesilir. Sistem ayakta kalıyor mu, kullanıcı fark ediyor mu (Basiri vd., 2016).
- **Deterministik simülasyon:** bütün sistem tek süreçte, sahte saat ve sahte ağla koşturulur; binlerce rastgele arıza senaryosu taranır. Bulunan hata aynı tohumla (*seed*) yeniden üretilebilir. Dağıtık sistemler için en güçlü tekniktir; kurulumu ağırdır.
- **Coverage-guided fuzz:** rastgele girdi verilir, hangi girdinin yeni kod yolu açtığı izlenir, o yönde mutasyon üretilir. Saatlerce koşar, çökme arar. Ayrıştırıcılar ve dosya okuyucular için uygundur.

Üretim Trafiği Ailesi

- **Trafik tekrarı:** canlıdan gerçek istekler kaydedilir, yeni sürüme aynı istekler oynatılır. Senaryo uydurulmaz, gerçek kullanıcı davranışı kullanılır.
- **Gölge trafik:** canlı isteğin kopyası yeni sürüme de gönderilir. Kullanıcı eski sürümün cevabını alır; iki cevap karşılaştırılır. Risksiz canlı testtir.
- **Canary:** yeni sürüm kullanıcıların yüzde beşine verilir. Hata oranı artmıyorsa yüzde yirmi, elli, yüz. Artıyorsa geri alınır. Tek kişilik yerel ortamda anlamsızdır.

Güvenlik Ailesi

- **Penetrasyon:** güvenlik uzmanı saldırgan gibi sisteme girmeye çalışır. Elle, yaratıcı ve pahalıdır; yılda bir ya da büyük değişiklikte yapılır.
- **DAST:** çalışan uygulamaya otomatik saldırı taraması (SQL enjeksiyonu, XSS, bilinen açıklar). Penetrasyonun ucuz otomatik hâlidir.
- **SAST:** kodu okuyarak açık kalıbı arar: kullanıcı girdisi doğrudan SQL'e gidiyor mu, şifre kodda yazılı mı. Lint'in güvenlik odaklı biçimidir.

İnsan ve Organizasyon Ailesi

- **Game day:** alarm bilerek çaldırılır. Ekip runbook'a bakıp doğru şeyi kaç dakikada yapıyor. Sistemi değil insanı test eder.
- **Alfa, beta, dogfooding:** alfa iç ekip kullanımı, beta seçilmiş dış kullanıcılar, dogfooding ürünün geliştirici tarafından her gün kullanılması. Otomatik testin göremediği "bu tuhaf" hissini insan verir.
- **Kitle testi:** dış kalabalık ürünü farklı cihaz ve ülkelerde dener, hata raporlar. Uyumluluk için ucuz yoldur.

Regülasyon Ailesi

- **KVKK/GDPR silme kanıtı:** kullanıcı silinmeyi talep ettiğinde verinin veritabanı, yedek, log, önbellek ve üçüncü taraf servislerden gerçekten silindiği kanıtlanır.
- **SOC2 / PCI denetimi:** dış denetçi erişim kontrolü, log tutma ve yedekleme için kanıt ister. Sistem değil süreç denetimidir; kurumsal müşteri talep ettiğinde yapılır.
- **WCAG resmi denetimi:** erişilebilirlik standardının (WCAG 2.1 AA) resmi kontrol listesi. Otomatik taramanın ötesinde uzman elle kontrol eder ve belge verir. Kamu ya da büyük kurum müşterisi zorunlu tuttuğunda yapılır.

Adım 9: Tedarik Zinciri Denetimleri

Bu grup, kurulumu neredeyse maliyetsiz olmasına rağmen, tek geliştiricili ve kısıtlı bağımlılık setine sahip bir sistemde nadiren somut bir hata yakalar. Listede bulunmaları, ihtiyaç doğduğunda ne oldukları bilinsin dıyedir.

Bağımlılık Açık Taraması

Kullanılan paketlerde bilinen güvenlik açığı olup olmadığına bakan taramadır. Projede yüzlerce paket bulunur; birinde açık bulunursa geliştiricinin kodu değişmese bile sistem açık hâle gelir. `npm audit` komutu `package-lock.json` dosyasındaki her paketi bilinen açıklar veritabanıyla karşılaştırır ve hangi paketin hangi sürüme yükseltilmesi gerektiğini bildirir. CI'da tek satır olarak durur; ciddi bir açık çıktığında build'i kırar. Go tarafında `govulncheck`, Python'da `pip-audit` aynı işi görür.

Lisans Denetimi

Kullanılan paketlerin lisansının projeye izin verip vermediğine bakan taramadır. Her paketin bir lisansı vardır: MIT ve Apache serbesttir; GPL, paketi kullanan kodun da açık kaynak olmasını gerektirir; AGPL bunu sunucuda çalıştırılan yazılıma da uygular. Lisanssız paket ayrı bir risktir. Denetim tüm bağımlılıkları tarayıp uyumsuz lisansı bildirir; `license-checker` ya da `npm-license-crawler` kullanılır. Ticari müşteriye teslim edilen bir üründe GPL lisanslı bir paketin sızması, müşteri için hukuki sorun demektir.

Bütünlük Kontrolü

Paket kurulumunda iki soru vardır. Birincisi: bugün kurulan paket, dün kurulanla aynı mı; sürüm gizlice değişmiş olabilir. İkincisi: bu paket gerçekten sahibinin yayınladığı mı; hesap çalınmış ve içine zararlı kod konmuş olabilir. Birinci soruyu `package-lock.json` cevaplar: her paketin tam sürümünü ve parmak izini kaydeder; `npm ci` komutu tam o kaydedileni kurar, farklıysa durur. İkinci soruyu paket imzası (*provenance*) cevaplar: paketin gerçekten o kişi tarafından o depodan yayımlandığı doğrulanır. Uygulama: lockfile depoya konur, CI'da `npm install` yerine `npm ci` kullanılır.

Kapsama Ölçümü

Testlerin kodun yüzde kaçına dokunduğunu ölçen sayıdır. Test koşarken hangi satırların çalıştığı işaretlenir; sonunda kodun hangi bölümüne hiç test dokunmadığı görülür. Yüzde yüz kapsama hata olmadığı anlamına gelmez: bir satırın çalışmış olması doğru çalıştığı anlamına gelmez, test hiçbir şey doğrulaması bile satır “kapsanmış” sayılır. Ajanlara yazdırılan sahte testler tam bunu üretir: yüksek kapsama, sıfır yakalama gücü. Doğru kullanım, oranı hedef olarak koalamak değil, test dokunmamış dosyayı bulmak için gösterge olarak okumaktır.

Kesit 1: Veri

Veri kalitesi çoğunlukla tek bir şeye indirgenir: canlı ortamda çalışan ve eşik aşıldığında uyarı veren bir sayaç (boş alan oranı yüzde beşi geçti mi, süre değerleri negatif mi, tarih biçimi bozuk mu). Bu indirgeme eksiktir. Veri, boru hattında dört farklı noktada sınanır ve her noktanın test türü farklıdır. Sayaç yalnızca dördüncü noktadır.

Veri Testinin Dört Kapısı

Boru hattı şöyle düşünülür: kaynak → giriş kapısı → ara aşamalar → çıkış kapısı → canlı. Her kapıda ayrı bir soru sorulur.

- **Giriş kapısı:** bu veri hattın kabul ettiği biçimde mi (doğrulama).
- **Aşamalar arası:** bir aşamanın verdiği, sonrakinin beklediği şey mi (veri sözleşmesi).
- **Çıkış kapısı:** sonuç kendi kurallarına uyuyor mu (değişmezler).
- **Canlı:** veri zamanla sessizce değişti mi (izleme).

İlk üçü testtir: CI'da koşar, deterministiktir, kırmızı ya da yeşildir. Dördüncüsü sayaçtır: istatistiktir, eşik aşınca uyarır. Test ile sayaç arasındaki fark, cevabın kesinliğidir; test “bu kayıt yanlış” der, sayaç “bu hafta bir şeyler değişti” der.

Giriş Doğrulama

Hatta giren her kayıt şemaya karşı kontrol edilir: zorunlu alan var mı, tip doğru mu, değer aralıkta mı, kodlama bozuk mu. JSON Schema, Zod ve Pydantic bu iş için yaygın araçlardır. Kural: geçersiz veri hatta girmez, kapıda reddedilir ve reddedilme sebebi loglanır. Sessizce düzeltilip içeri alınan veri, sonraki aşamalarda açıklanamayan hatalar üretir.

Bu adım, Adım 4'teki şema testinin veri tarafıdır. Oradaki test model cevabına bakar, buradaki test kullanıcı verisine bakar; mekanizma aynıdır, yön terstir.

Girdi Sınır Testleri

Şema doğrulaması “biçim doğru mu” sorusunu cevaplar; sınır testleri “biçim doğru ama miktar ya da içerik makul mü” sorusunu sorar. Her girdi türü için dört sınıf kötü girdi denenir.

Çok Büyük Girdi

Beklentinin çok üstünde boyut: on gigabaytlık video, on saatlik ses, yüz bin satırlık liste, on megabaytlık JSON. Sistem ya kapıda reddeder (üst sınır tanımlı ve mesaj net) ya da kabul edip bitirir; kabul edip yarım bırakması ya da belleği

doldurup çökmesi hatadır. Bu testler yavaş olabilir; o yüzden üst sınırın kendisi ve sınırdan bir fazlası test edilir, arası değil. Bu, sınır değer analizinin dosya boyutuna uygulanmış hâlidir. Zip-bomb bu sınıfın özel bir üyesidir: küçük görünüp büyük açılan girdi.

Çok Küçük ya da Boş Girdi

Sıfır baytlık dosya, boş liste, tek karelik video, tek karakterlik metin. Sistem çoğu zaman “normal” veri için yazılmıştır; boş dizide indeks alma, sıfıra bölme, ilk elemanı varsayma hataları burada çıkar.

Bozuk Girdi

Uzantı doğru içerik yanlış (adı .mp4, içeriği metin), başlığı sağlam gövdesi kesik dosya, iç içe bin seviye JSON, geçersiz UTF-8, kodeki desteklenmeyen video, yarıda kesilmiş yükleme. Her biri için beklenen davranış tek tiptir: hat çökmez, takılı kalmaz, düzgün bir hata döner ve sebebi loglanır. Bozuk medya korpusu (Adım 7) bu testlerin kalıcı deposudur; bulunan her yeni bozuk dosya korpusa eklenir.

Yavaş Girdi

Boyut normal ama gelişi yavaş: yükleme saniyede bir kilobayt gidiyor ya da ortada duruyor. Zaman aşımı var mı, yarım dosya diskte kalıyor mu. Arıza enjeksiyonuna yakındır ama girdi kaynaklıdır; ağ kesitiyle kesişir.

Aşamalar Arası Veri Sözleşmesi

Boru hattındaki her aşama bir üretici ve bir tüketicidir. Ayrıştırıcı “klip listesi” üretir, birleştirici onu tüketir. Sözleşme yazılı hâle getirilir: alan adları, tipler, birimler (saniye mi milisaniye mi), boş değer anlamı. Test iki yönlüdür: üretici test edilirken çıktısı sözleşmeye karşı denetlenir; tüketici test edilirken sözleşmeye uyan bir fixture verilir. İki taraf aynı fixture’ı paylaşırsa, biri değişip diğeri değişmediğinde test kırılır.

Bu, Adım 4’teki sözleşme testinin hattın içine taşınmış hâlidir. Oradaki sözleşme dış tarafla (model) yapılır; buradaki sözleşme iç aşamalar arasındadır. Birim ve saniye–milisaniye karışıklığı, iç sözleşmesi yazılmamış hatların en sık hatasıdır.

Çıkış Değişmezleri

Sonucun doğru olup olmadığı bilinmese bile uyması gereken kurallar vardır. Klip için: bitiş başlangıçtan büyüktür, hiçbir klip video süresini aşmaz, klipler çakışmaz, kimlikler benzersizdir, her klip gerçekten var olan bir kaynağa bağlıdır. Bunlar her koşuda otomatik kontrol edilir; ihlal eden çıktı hat dışına çıkmaz.

Adım 7'deki faturalama değişmeziyle aynı mantıktır; orada para korunur, burada veri. Adım 4'teki metamorfik test de buraya bağlanır: girdi değişince çıktının nasıl değişmesi gerektiği, girdiye bağlı bir değişmezdir.

Hat Bütünlüğü

Tek tek kayıt değil, hattın tamamı sınanır.

- **Sayım:** giren kayıt sayısı, çıkan artı reddedilen artı hatalı toplamına eşit mi. Sessizce kaybolan kayıt en sık hata sınıfıdır ve yalnızca sayımla görülür.
- **Yarım kalma:** hat ortada kesilirse ne olur; yarım kayıt kalıyor mu, tekrar başlayınca çift oluşuyor mu. Arıza enjeksiyonunun veri odaklı biçimidir.
- **Soy izi** (*lineage*): her çıktı kaydı hangi girdiden, hangi sürümle, ne zaman üretildi. Test değil, hata bulunduğu geriye yürüebilmek için tutulan kayıttır.

Determinizm

Aynı girdi, aynı kod, aynı ayar: çıktı aynı olmalıdır. Test basittir: hat aynı girdiyle iki kez koşturulur, iki çıktı bayt bayt karşılaştırılır. Fark varsa üç sebepten biridir.

- **Rastgelelik:** sıralama garantisi olmayan yapılar (Go'da `map` iterasyonu, paralel işlenip birleşen listeler), rastgele kimlik üretimi, zaman damgası. Çözüm ya sıralamayı sabitlemek ya da karşılaştırmadan önce bu alanları çıkarmaktır: kimlik ve zaman damgası kırılır, liste sıralanır, sonra karşılaştırılır.
- **Dış kaynak:** model çağırısı, saat, ağ. Test fixture ile koşuyorsa bunlar sabittir; canlı modelle koşuyorsa determinizm beklenemez ve test bu moda uygulanmaz.
- **Gerçek hata:** paylaşılan durum, temizlenmemiş önbellek, önceki koşudan kalan geçici dosya. En değerli bulgu budur; ikinci koşu birinciden farklıysa hat kendi artığını okuyor demektir.

Bu testin iki türevi vardır. **Yeniden koşma** testi, hattın aynı girdiyle ikinci kez çalıştırıldığında veritabanında ikinci bir kayıt üretmediğini sınar; bu idempotency'nin veri hâlidir. Determinizm çıktının aynı olmasını, idempotency yan etkinin tekrarlanmamasını ister. **Ortam paritesi** ise aynı girdinin yerel makinede ve CI'da aynı çıktıyı vermesidir; farklıysa sebep genellikle kodek sürümü, yerel ayar (*locale*) ya da saat dilimidir.

Determinizm testi altın veri setinin ön koşuludur: hat deterministik değilse altın setle karşılaştırma her koşuda sebepsiz kırmızı olur ve titrek test sınıfına düşer.

Altın Veri Seti

Elle doğrulanmış, küçük ve sabit bir girdi-çıkı çifti seti tutulur: on video, her biri için “doğru” klip listesi. Hat her sürümde bu set üzerinde koşar, sonuç kayıtlarla karşılaştırılır. Adım 1’deki characterization testinden farkı, kaydın “kod ne yapıyor” değil “ne yapmalı” olmasıdır; birisi elle kontrol etmiştir. Bu set, veri tarafının regresyon setidir ve zamanla büyür: canlıda yaşanan her veri hatası, sebep olan girdiyle birlikte sete eklenir.

Canlı İzleme

Sayaç burada kalır, ancak ölçülen şey netleştirilir. Klasik veri kalitesi boyutları şunlardır: tamlık (boş alan oranı), geçerlilik (aralık dışı değer), benzersizlik (çift kayıt), tutarlılık (iki tablo birbirini tutuyor mu), güncellik (son kayıt ne zaman geldi), hacim (bugün beklenenin çok altında ya da üstünde mi geldi). Her boyut için bir taban çizgisi alınır ve sapma ölçülür.

Buna kayma (*drift*) izleme denir: hiçbir kayıt tek başına bozuk değildir ama dağılım değişmiştir. Ortalama klip süresi bir haftada on saniyeden dört saniyeye inmişse model sessizce güncellenmiş olabilir. Bu, Adım 4’teki kurallı eval’in canlı trafik üzerinde sürekli koşan hâlidir.

Döngüdeki Yeri

Veri kesiti tek bir adıma sığmaz. Giriş doğrulama ve aşama sözleşmesi Adım 4’ün yanına gider (fixture tabanlı, hızlı); çıkış değişmezleri ve hat bütünlüğü Adım 7’ye (sistemde veri hattı varsa); altın set Adım 5’e (regresyon); canlı izleme sentetik izlemenin yanına. Kesitin tek bir cümlelik özeti şudur: veri kalitesi bir sayaç değil, hattın her kapısında ayrı bir test ailesidir; sayaç yalnızca son kapıdır.

Kesit 2: Ağ

Ağ, veri hattının hem giriş hem çıkış tarafında durur: kullanıcı veriyi ağ üzerinden yükler, hat modeli ağ üzerinden çağırır, sonuç ağ üzerinden indirilir. Bu yüzden ağ testi üç kesitte incelenir: ağın hangi bozukluğu deniyor, bozukluk hattın hangi noktasında yaşanıyor, sistemden ne bekleniyor. Nygard (2007), üretim sistemlerinde yaşanan arızaların büyük kısmının ağ sınırlarında ve ayarlanmamış zaman aşımalarında çıktığını, çözümün bu sınırlara bilinçli kırılma noktaları koymak olduğunu belirtmiştir.

Ağ Bozukluğunun Türleri

Ağ bozukluğu tek bir şey değildir; en az altı farklı davranış vardır ve her biri farklı hata üretir.

- **Kopma:** bağlantı aniden kesilir. Sistem hatayı alır; ama ne kadar sürede aldığı ve o ana kadar ne yaptığı önemlidir.
- **Gecikme:** paket gider ama geç varır (200 ms, 2 s, 30 s). Zaman aşımı ayarlarını ve bekleyen istek birikmesini test eder.
- **Kayıp:** paketlerin bir kısmı (yüzde 1, 5, 30) hiç varmaz. TCP yeniden gönderir ama bağlantı yavaşlar ve büyük aktarımlar çok uzar.
- **Dar bant:** hız düşer (3G, uydu, paylaşımlı Wi-Fi). Bir gigabaytlık yükleme saatler sürer; kullanıcı arayüzü bu süreyi nasıl gösterir.
- **Kesik kesik:** bağlantı gidip gelir. En sinsili olanıdır; kopmadan farkı, her koptuğunda yeniden gelmesi ve sistemin “geldi, koptu, geldi” döngüsüne girmesidir.
- **Kısmi arıza:** bağlantı var, DNS var, TLS el sıkışması oluyor ama karşı taraf cevap vermiyor ya da yarım cevap veriyor (HTTP 200 dönüp gövdenin kesilmesi). “Ağ çalışıyor” görünür ama istek tamamlanmaz.

Bunlar dışında iki altyapı arızası vardır. DNS çözümlemesi başarısız olur ya da yanlış adrese gider; TLS sertifikası geçersiz ya da süresi dolmuştur. İkisi de “ağ var ama bağlanılamıyor” durumudur ve uygulama katmanında ağ hatası gibi değil, farklı hata sınıfları olarak görünür.

Bozukluğun Hattaki Yeri

Aynı bozukluk hattın üç yerinde farklı sonuç doğurur.

Yükleme Tarafı (Kullanıcı → Sistem)

Yükleme ortada kopunca yarım dosya diskte kalıyor mu, kayıt “yüklendi” olarak işaretlenmiş mi. Kesik bağlantıda yükleme devam ettirilebiliyor mu (*resumable upload*) yoksa baştan mı başlıyor. Dar bantta ilerleme çubuğu doğru mu, zaman

aşımı yüklemeyi bitmeden öldürüyor mu. Kullanıcı yükleme sırasında sekmeyi kapatırsa sunucu tarafında ne kalıyor.

Model Tarafı (Sistem → Model Sağlayıcısı)

En kritik kesittir, çünkü ücretlidir ve yavaştır. Çağrı zaman aşımına uğrarsa yeniden deniyor mu, kaç kez, ne aralıkla. Yeniden deneme aynı işi iki kez ücretlendiriyor mu. Cevap yarım gelirse (akış ortasında kesilen *stream*) ayrıştırıcı ne yapıyor. Sağlayıcı 429 (çok fazla istek) ya da 503 dönerse hat bekliyor mu, düşüyor mu. Gecikme otuz saniyeyi aşınca kullanıcıya ne gösteriliyor. Model çağrısı sırasında kopan ağ, Adım 4'teki bozuk fixture testinin canlı versiyonudur.

İndirme Tarafı (Sistem → Kullanıcı)

Büyük sonuç dosyası indirilirken kopunca kaldığı yerden devam edilebiliyor mu (HTTP **Range** isteği). WebSocket bağlantısı koparsa istemci yeniden bağlanıyor mu; bağlanınca kaçırdığı mesajları alıyor mu, yoksa “işlem bitti” haberi kaybolup kullanıcı sonsuza kadar bekliyor mu.

İç Ağ

Sistem kendi veritabanı, kuyruğu ve nesne depolamasıyla (S3, MinIO) ağ üzerinden konuşur. Veritabanına giden bağlantı kopunca bağlantı havuzu (*connection pool*) toparlanıyor mu, kuyruğa yazılamayan iş kayboluyor mu, S3'e yarım yazılan nesne bozuk kalıyor mu. Bu kesit çoğu zaman unutulur, çünkü yerel ortamda hiç kopmaz.

Sistemden Beklenen Davranış

Her ağ testinin geçme ölçütü şu dört sorudur.

1. **Hata görünür mü:** sistem düzgün bir hata döner ve loglar; sessizce yanlış sonuç vermez, sonsuza kadar beklemez.
2. **Durum tutarlı mı:** yarım kayıt, yarım dosya, “işleniyor”da takılı kalan iş yoktur. Ya tamamlanmıştır ya hiç başlamamış gibidir.
3. **Toparlanıyor mu:** ağ geri gelince sistem kendiliğinden devam eder; yeniden başlatma gerekmez, havuz ve bağlantılar kendini onarır.
4. **Tekrar güvenli mi:** yeniden deneme çift kayıt, çift ücret, çift e-posta üretmez (idempotency).

Nasıl Test Edilir

Ağ testleri katmana göre üç araçla yapılır; dördüncüsü canlıdır.

Birim ve Bileşen Katmanı

Ağ sahte olduğundan bozukluk kodla üretilir. Sahte HTTP sunucusu (JavaScript'te `msw` ya da `nock`, Go'da `httptest`) belirli isteğe gecikmeli, yarım ya da hatalı cevap döndürecek şekilde ayarlanır. Ucuz ve hızlıdır; yeniden deneme mantığı, zaman aşımı değeri ve hata yolu burada test edilir. Bozuk fixture testinin ağ tarafıdır: fixture'ı bozmak yerine fixture'ın gelişi bozulur.

Sınır Katmanı

Gerçek ağ üzerinde, bozukluk işletim sistemi ya da vekil (*proxy*) seviyesinde üretilir. Linux'ta `tc` ve `netem` gecikme, kayıp ve bant sınırı ekler; Toxiproxy iki servis arasına girip bağlantıyı test sırasında keser, geciktirir ya da yarım bırakır; Docker ağı test sırasında koparılabilir. Bu katmanda veritabanı ve kuyruk kopmaları test edilir. Arıza enjeksiyonunun (Adım 7) ağ odaklı hâlidir.

Sistem Katmanı

Tarayıcıdan koşan uçtan uca testte ağ profili ayarlanır. Playwright ve Chrome DevTools “Slow 3G” ve “Offline” profillerini ve isteğe özel gecikme sağlar; test ortasında bağlantı kapatılıp açılabilir. Yükleme ve indirme senaryoları burada koşar.

Canlı

Sentetik izleme farklı coğrafyalardan ve farklı ağ kalitelerinden yapılırsa, kullanıcının gerçekten gördüğü ağ koşulu ölçülür. CI'daki tüm ağ testleri kontrollü bozukluktur; gerçek ağın nasıl bozulduğunu yalnızca canlı gösterir.

Test Matrisi

Ağ testleri karar tablosu gibi kurulur: satırlar bozukluk türü (kopma, gecikme, kayıp, dar bant, kesik, kısmi), sütunlar hattaki yer (yükleme, model, indirme, iç ağ). Her hücrede beklenen davranış yazılır.

Bozukluk	Yükleme	Model	İndirme	İç ağ
Kopma	yarım dosya silinir, kayıt “yüklenmedi”	yeniden deneme, tek ücret	Range ile devam	havuz onarılır
Gecikme	ilerleme doğru, zaman aşımı yok	30 s üstünde kullanıcıya bilgi	indirme sürer	sorgu zaman aşımı tanımlı
Kayıp	yükleme uzar, bitince tam	yeniden deneme	indirme uzar, bitince tam	yeniden bağlanma
Dar bant	devam ettirilebilir yükleme	—	devam ettirilebilir indirme	—
Kesik	baştan başlamaz	döngüye girmez	WebSocket yeniden bağlanır, mesaj kaçırmaz	kuyruk işi kaybetmez
Kısmi	kesik gövde reddedilir	yarım stream düzgün hata	—	yarım nesne görünmez

Tablo dolduğunda iki şey görünür: hangi hücrenin hiç düşünülmediği ve hangi hücrelerde beklenen davranışın aynı olduğu; ikinci gruptaki hücreler tek testle kapanır. Tablonun tamamı test edilmez. Sistemde gerçekten yaşanan hücreler önce yazılır, gerisi ilk arıza yaşandığında açılır (Adım 8 mantığı).

Yaygın Hatalar

Bu testlerin bulduğu tipik hata sınıfları şunlardır: zaman aşımı hiç ayarlanmamış (varsayılan sonsuz); yeniden deneme aralığı sabit ve tüm istemciler aynı anda tekrar vuruyor (üstel geri çekilme yok); yeniden deneme sayısı sınırsız; kopan WebSocket yeniden bağlanmıyor ya da bağlanıp eski oturumu bilmiyor; bağlantı havuzu ölü bağlantıyı tutup her istekte hata veriyor; yarım yükleme “tamamlandı” sayılıyor; hata mesajı kullanıcıya “bir şeyler ters gitti” dışında bilgi vermiyor. Bunların hiçbiri normal ağda görünmez; hepsi yalnızca ağ bozulduğunda çıkar ve bozulduğunda hepsi aynı anda çıkar.

Kesit 3: Durum ve Depolama

Veri hattın içinden akan şeydir; durum, duran şeydir. Veritabanı, dosya sistemi, nesne depolama (S3, MinIO), önbellek ve kuyruk bu kesitin öğeleridir. Notta migrasyon, yedek ve önbellek doğruluğu tek tek yer alır; bu kesit onları tek bir soru altında toplar: durum bozulduğunda ya da eşzamanlı değiştirildiğinde sistem ne yapıyor.

Eşzamanlı Yazma

Aynı kayda iki işlem aynı anda yazınca ne olur. Cevap veritabanının izolasyon seviyesine bağlıdır ve varsayılan seviye çoğu zaman geliştiricinin sandığından zayıftır. Test: iki işlem aynı satırı okur, ikisi de değeri artırır, ikisi de yazar; sonuçta artış bir mi iki mi. Kredi defteri, sayaç ve kuyruk sırası bu testin klasik konularıdır. Adım 3'teki eşzamanlılık birim testi bu sınıfı sahte veriyle, bu kesit gerçek veritabanıyla bulur; ikisi farklı hata yakalar, çünkü yarış gerçek zamanlamada farklı biçimde oluşur.

Yarıda Kalan İşlem

İşlem ortada kesilirse geri alınıyor mu. Test: bir işlem iki tablo yazacakken ilk yazmadan sonra süreç öldürülür; ikinci tablo boş, birinci tablo dolu kalıyor mu. Kesme, veritabanı işlemi (*transaction*) içinde yapılırsa geri alınır; işlem dışında yapılırsa yarım kayıt kalır. Dosya sistemi ve nesne depolamada işlem kavramı yoktur; orada yarım yazılan dosya diskte durur ve “geçici ad ile yaz, sonra yeniden adlandır” kalıbının kullanılıp kullanılmadığı test edilir.

Kuyruk Teslim Garantisi

Kuyruktaki iş işlenirken süreç ölürse iş kayboluyor mu, tekrar mı geliyor. Kuyruk sistemleri iki garanti sunar: en az bir kez (*at-least-once*, iş tekrar gelebilir, işleyici idempotent olmalıdır) ve en fazla bir kez (*at-most-once*, iş kaybolabilir). Tam bir kez garantisi pratikte en az bir kez artı idempotent işleyici demektir. Test: iş alınır, onaylanmadan süreç öldürülür, işin geri gelip gelmediğine bakılır; sonra iş iki kez teslim edilir, sonucun bir kez işlenmiş gibi olup olmadığına bakılır.

Disk Dolması

Disk dolunca hangi işlem ilk düşer. Test: disk bilerek doldurulur (Adım 7'deki arıza enjeksiyonu), yükleme, log yazma ve veritabanı yazma sırayla denir. Beklenen: yükleme düzgün hata verir, log yazamaması sistemi durdurmaz, veritabanı bozulmaz. Sık görülen hata, log yazamayan sürecin çökmesidir; gözlem aracının ana işi öldürmesi ters bir sonuçtur.

Nesne Depolama Görünürlüğü

S3 benzeri depolamaya yarım yazılan nesne okuyucuya görünür mü. Çoğu nesne depolama nesneyi yazma tamamlanana kadar göstermez; ancak çok parçalı yükleme (*multipart upload*) yarıda kalırsa parçalar depoda kalır ve ücret üretir. Test: çok parçalı yükleme ortada kesilir; nesne listelemede görünmüyor mu, yarım parçalar temizleniyor mu.

Canlı Migrasyon Kilidi

Büyük tabloya sütun eklemek ya da indeks kurmak bazı veritabanlarında tabloyu kilitler; kilit süresince tüm yazmalar bekler. Test, sahne ortamında gerçek boyutta veriyle migrasyon koşturup sürenin ve kilidin ölçülmesidir. Adım 7'deki migrasyon testi verinin sağlam kalıp kalmadığına bakar; bu test migrasyonun sistemi durdurup durdurmadığına bakar.

Döngüdeki Yeri

Bu kesit veri ile ağın kesişimidir: veritabanı ağın öbür ucundadır ve durumu tutar. Testlerin tamamı sınır katmanında (gerçek veritabanı, gerçek depolama, tarayıcı yok) koşar ve Adım 7'nin entegrasyon başlığına bağlanır.

Kesit 4: Zaman

Adım 3'teki saat ve saat dilimi alt başlığı zamanın yalnızca bir yüzüdür. Zaman kesiti, sistemin süreye, sıraya ve süre dolmasına bağlı tüm davranışlarını kapsar.

Uzun Süren İş ve Süreç Yeniden Başlaması

On dakikalık video işleme sırasında süreç yeniden başlarsa ne olur. İş kaldığı yerden mi devam eder, baştan mı başlar, yoksa “işleniyor” durumunda sonsuza kadar mı kalır. Üçüncüsü en sık görülen hatadır: iş durumu veritabanında “işleniyor” olarak kalır, onu işleyen süreç yoktur ve hiçbir şey onu temizlemez. Test: iş başlatılır, süreç öldürülür, sistem yeniden başlatılır; işin durumu ve son davranışı gözlenir. Beklenen: ya iş yeniden kuyruğa girer ya da bir bekçi belirli süre sonra “işleniyor”da takılı işleri başarısız olarak işaretler.

Zamanlanmış İşlerin Çift Tetiklenmesi

Gece çalışan temizlik işi iki kez tetiklenirse ne olur. İki sunucu varsa ikisi de aynı işi çalıştırır; tek sunucuda süreç yeniden başlarsa iş tekrar tetiklenebilir. Test: zamanlanmış iş bilerek iki kez, örtüşen zamanlarda çalıştırılır; iki kez silme, iki kez rapor gönderme, iki kez ücretlendirme oluyor mu. Çözüm genellikle dağıtık kilit ya da işin idempotent yazılmasıdır.

Süre Dolması

Yetki belirteci, imzalı URL, geçici dosya, oturum: her birinin bir ömrü vardır. Test, tam sınırdan yapılır: belirteç bir saniye önce geçerli, bir saniye sonra geçersiz mi. Ayrıca süre dolması ile devam eden işlem arasındaki çakışma sınırı: on dakikalık yükleme sırasında oturum dolarsa yükleme kaybediliyor mu, imzalı URL dosya indirilirken dolarsa indirme yarım mı kalıyor.

Sıra Boşluğu

Birbirine bağlı iki adım arasında boşluk vardır: dosya depoya yüklendi ama kayıt henüz veritabanına yazılmadı; kayıt yazıldı ama önbellek henüz güncellenmedi. Bu boşlukta gelen istek ne görür. Test: iki adım arasına bilerek gecikme konur ve o anda okuma yapılır. Beklenen: ya tutarlı bir “henüz yok” cevabı ya da ikinci adım tamamlanana kadar birincinin görünmemesi. Yarım durumun kullanıcıya görünmesi hatadır.

Nasıl Test Edilir

Birim düzeyinde sahte saat kullanılır: sistem saati test tarafından ilerletilir, süre dolmaları anında tetiklenir, yaz saati geçişi tek satırla kurulur. Sahte saat olmadan zaman testleri gerçek zamanda bekler ve titrek olur. Süreç ölümü ve yeniden başlama sınır katmanında test edilir; orada sahte saat yoktur, gerçek

süreç öldürölür. Adım 8'deki deterministik simölasyon, zaman kesitinin en güçlü aracıdır: tüm sistem sahte saatle koşar ve binlerce zamanlama senaryosu taranır; kurulumu ağır olduđu için ilk zamanlama arızasına kadar kapalı tutulur.

Ek Kesitler

Aşağıdaki kesitler daha kısa tutulur. Bir kısmı mevcut adımlara birer paragraf olarak eklenir, bir kısmı Adım 8 mantığıyla ilk arıza yaşandığında açılır.

Kimlik

Yetki matrisi ve kiracı izolasyonu “kim ne yapabilir” sorusunu cevaplar; kimlik kesiti “kimlik değişince ne olur” sorusunu sorar. Oturum süresi dolunca yarıda kalan işlem ne oluyor. Şifre değişince diğer cihazlardaki oturumlar düşüyor mu. Kullanıcı silinince verisi ve çalışan işleri ne oluyor: çalışan iş sahipsiz kalıyor mu, veri gerçekten siliniyor mu (Adım 8’deki KVKK silme kanıtı). Aynı kullanıcı iki sekmede çıkışan işlem yaparsa. API anahtarı iptal edilince hâlâ çalışan istek tamamlanıyor mu, kesiliyor mu. Bu testler yetki matrisinin yanına, Adım 7’ye eklenir.

Kaynak Sınırları

Ağ bir kaynaktır; bellek, işlemci, disk, dosya tanıtıcısı, bağlantı sayısı ve iş parçacığı da öyle. Kaynak temizliği (Adım 3) ve soak testi (Adım 8) bu kesitin parçalarıdır, ancak tek bir soru altında toplanmaz: her kaynağın üst sınırı tanımlı mı; sınırı vurunca sistem ne yapıyor (reddediyor mu, yavaşlıyor mu, çöküyor mu); kaynak geri gelince toparlanıyor mu. Veri kesitindeki büyük girdi testi bu kesitin girdi tarafıdır; kaynak sınırı testi sistem tarafıdır. Bellek sınırı test edilirken süreç sınırlı bellekle (Docker `--memory`) koşturulur ve büyük girdi verilir; beklenen, sınırı yaklaşıncı düzgün reddetmesidir, işletim sisteminin süreci öldürmesi değil. Bu kesit ilk kaynak arızasında açılır.

Yapılandırma ve Ortam

Aynı kod farklı yapılandırmayla farklı sistemdir. Eksik ortam değişkeni açılışta mı yoksa ilk kullanımda mı patlıyor; açılışta patlaması tercih edilir, çünkü ilk kullanım gece üçte olabilir. Yanlış tipte değer (metin yerine sayı) nasıl yakalanıyor. Yerel, sahne ve canlı arasındaki farklar (kodek sürümü, yerel ayar, dosya yolu) belgeli mi. Özellik bayrağıyla (*feature flag*) açılıp kapanan özellikler her iki durumda da test ediliyor mu; genellikle yalnızca açık hâli test edilir ve kapalı hâl bozular. Gizli anahtar kodda ya da logda sızıyor mu (Adım 8’deki SAST). Doküman testi (Adım 6) bu kesitin bir üyesidir. Yapılandırma doğrulaması açılışta tek bir şema kontrolüyle yapılır ve Adım 0’a eklenir.

Gözlemlenebilirlik

Adım 2’de log içeriği test edilmez denmiştir; logun varlığı ve işlevi ise test edilebilir. Üç test vardır. Birincisi: her hata yolu en az bir log satırı üretiyor mu; fitness function ile denetlenir. İkincisi: istek kimliği (*correlation ID*) hattın

başından sonuna taşınıyor mu; bir isteğin tüm log satırları tek kimlikle bulunabiliyor mu. Üçüncüsü: alarm gerçekten çalışıyor mu; bilerek hata üretilir, alarm geldi mi (Adım 8'deki game day'in küçük hâli). Bir dördüncü test daha vardır: bilinen bir hata enjekte edilir, yalnızca loga bakan biri sebebi bulabiliyor mu. Bu, logun “var” olmasını değil “yeterli” olmasını sınar.

Dağıtım ve Geri Alma

Smoke ve canary dağıtımın sonrasını sınar; dağıtımın kendisi ayrı bir kesittir. Yeni sürüm yüklenirken eski sürüm hâlâ istek alıyor mu (sıfır kesinti). İki sürüm aynı anda çalışırken veritabanı şeması ikisiyle de uyumlu mu; şema değişikliği önce eklemeli, sonra kod, en son silmeli yapılmazsa eski sürüm yeni şemada çöker. Geri alma gerçekten çalışıyor mu; en az bir kez sahne ortamında denenmiş olmalıdır, ilk gerçek ihtiyaçta denenmesi Adım 7'deki yedekten dönüş ile aynı hatadır. Dağıtım ortada kesilirse sistem hangi durumda kalıyor. Adım 7'deki migrasyon ve geri dönüş bu kesitin veri parçasıdır; kod parçası buraya aittir.

Kullanıcı Hatası

Ağ ve veri dışarıdan gelen bozukluktu; kullanıcı da dışarıdan gelen bozukluktur. Yanlış sırayla tıklama, geri tuşu, form ortasında sayfa yenileme, aynı düğmeye on kez basma, iki sekmede aynı işi başlatma, yükleme bitmeden çıkma, yanlış dosyayı yükleyip iptal etme. Keşif testi (Adım 6) bunları elle bulur; en sık olanlar (çift tıklama, geri tuşu, yenileme) otomatik uçtan uca sete alınır. İdempotency (Adım 3) çift tıklamanın sunucu tarafıdır; bu kesit arayüz tarafıdır: düğme ikinci tıklamada devre dışı kalıyor mu, yenileme sonrasında form durumu korunuyor mu.

Kapsam Dışı Testler

Aşağıdaki türler haritanın tamlığı için listelenir; incelenen sistem türü için uygulanmaz.

- **Biçimsel doğrulama** (TLA+): algoritmanın matematiksel modellenip “bu kilitlenme asla olamaz” biçiminde ispatlanması. Dağıtık sistem çekirdekleri için kullanılır.
- **Sembolik ve concolic yürütme**: kodun cebirsel olarak çalıştırılıp her dala ulaşan girdinin denklemlerle çözülmesi. Akademik ağırlıklıdır.
- **Mobil senaryolar**: akış ortasında telefonun çalması, pil tüketimi, zayıf ağ profilleri, işletim sistemi izinlerinin geri alınması.
- **Donanım testleri**: gerçek kartın döngüye takılması (HIL/SIL), sıcaklık ve titreşim dayanımı, bit-flip dayanımı.

Sonuç

Test türlerinin sayısı sınırsız görünse de büyük çoğunluğu dört sorunun cevabından türetilir: sistemin ne kadarı çalışıyor, girdiyi kim üretiyor, “geçti” kararı nasıl veriliyor, hangi hata sınıfı avlanıyor. Regresyon, smoke, fuzz ya da AI eval gibi türler bu dört cevabın bileşimidir; yeni bir tür adı duyulduğunda aynı dört soru sorularak haritadaki yeri bulunabilir. Bu çerçevenin pratik sonucu, niyet sütununda boş kalan satırın gerçek boşluk olmasıdır: sistemde yaşanan ama yakalayıcısı bulunmayan hata sınıfı.

Kesit ekseni bu haritaya üçüncü bir boyut ekler ve boşlukları farklı bir yerden gösterir. Adımlar zamana göre düzenlenmiştir: hangi test ne zaman yazılır. Kesitler sisteme göre düzenlenmiştir: hangi boyut hiç sınanmamış. Bir sistemin bin testi olup ağ kesitinde hiç testi olmaması mümkündür; adım listesine bakan bunu görmez, kesit listesine bakan görür. Kesitlerin ortak dersi, aynı bozukluğun her katmanda başka bir araçla test edilmesi gerektiğidir: ağ kopması birim katmanında sahte sunucuyla, sınır katmanında Toxiproxy ile, sistem katmanında tarayıcı profiliyle, canlıda coğrafi izlemeyle sınanır ve bunlardan birinin varlığı diğerlerinin yerini tutmaz.

Döngünün kendisi, hazır malzemeden başlayıp maliyetli olana doğru ilerlediği için hızlı ekipler için uygundur; kitabı sıranın (önce test ağı, sonra değişiklik) tersine kurulmuş olması, mevcut bir birim test tabanının ağ görevi görmesiyle telafi edilir. Sıfırdan başlayan ve hiç testi olmayan bir projede aynı döngü tehlikelidir; orada teknik borç adımıdan önce en azından küçük bir uçtan uca set kurulmalıdır. Kesitler için de aynı uyarı geçerlidir: sekiz kesitin tamamı ilk döngüde yazılırsa not kendi kuralını ihlal eder. Hızlı ekip için sıralama şudur: veri ve ağ ilk döngüde, durum ve zaman ikinci döngüde, gözlemlenebilirlik ve dağıtım kısa paragraflar olarak mevcut adımlara, kaynak sınırları ve yapılandırma ilk arızada, kimlik ve kullanıcı hatası mevcut testlere birer ek olarak.

Üç sınırlılık açık kalmaktadır. Birincisi, ajanlara yazdırılan testlerin gerçek yakalama gücü yalnızca mutasyon testiyle ölçülebilir ve mutasyon testi büyük kod tabanlarında saatler sürer; hangi modüllerde koşturulacağının seçimi bu notun kapsamı dışındadır. İkincisi, yapay zekâ modeliyle konuşan uçlar için kurallı eval’ler yalnızca mekanik nitelikleri yakalar; özet kalitesi gibi yargı gerektiren nitelikler için yargıç modelin kendi tutarsızlığı henüz çözülmüş bir problem değildir. Üçüncüsü, ağ ve zaman kesitlerindeki testlerin tamamı kontrollü bozukluk üretir; gerçek ağın ve gerçek zamanın nasıl bozulduğu CI’da tam olarak taklit edilemez, bu yüzden bu kesitlerde canlı izleme testin tamamlayıcısı değil, zorunlu parçasıdır.

Kaynaklar

- Bach, J. (2000). Session-Based Test Management. *Software Testing and Quality Engineering*.
- Basiri, A., Behnam, N., de Rooij, R., Hochstein, L., Kosewski, L., Reynolds, J. ve Rosenthal, C. (2016). Chaos Engineering. *IEEE Software*, 33(3).

- Chen, T. Y., Cheung, S. C. ve Yiu, S. M. (1998). Metamorphic Testing: A New Approach for Generating Next Test Cases. Teknik rapor, Hong Kong University of Science and Technology.
- Claessen, K. ve Hughes, J. (2000). QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. *ICFP 2000*.
- Cohn, M. (2009). *Succeeding with Agile*. Addison-Wesley. (Test piramidi kavramının kaynağı.)
- DeMillo, R. A., Lipton, R. J. ve Sayward, F. G. (1978). Hints on Test Data Selection: Help for the Practicing Programmer. *IEEE Computer*, 11(4).
- Feathers, M. (2004). *Working Effectively with Legacy Code*. Prentice Hall. (Characterization testi kavramının kaynağı.)
- Ford, N., Parsons, R. ve Kua, P. (2017). *Building Evolutionary Architectures*. O'Reilly. (Fitness function kavramının kaynağı.)
- Fowler, M. (2011). Eradicating Non-Determinism in Tests. *martinfowler.com*.
- Fowler, M. (2012). TestPyramid. *martinfowler.com*.
- Kuhn, D. R., Wallace, D. R. ve Gallo, A. M. (2004). Software Fault Interactions and Implications for Software Testing. *IEEE Transactions on Software Engineering*, 30(6).
- Meszaros, G. (2007). *xUnit Test Patterns*. Addison-Wesley.
- Nygard, M. T. (2007). *Release It!* Pragmatic Bookshelf. (Zaman aşımı, devre kesici ve ağ sınırı kalıplarının kaynağı.)